

2 Description Detaillee du Modele et Processus d’Entrainement

Cette section fournit un compte rendu exhaustif et rigoureux de l’architecture algorithmique retenue, de la modelisation mathematique du probleme de detection, ainsi que du protocole d’entrainement mis en oeuvre. L’objectif est de garantir la parfaite reproductibilite de nos travaux, conformement aux exigences methodologiques enoncees.

2.1 Formalisation du Probleme et Structure de la Grille

Le projet vise a detecter et classifier neuf types distincts de ballons et balles de sport ($N_c = 9$). Les etiquettes de classes definies sont les suivantes :

```
['americanFootball', 'ball_pingpong', 'ball_tennis', 'ballon_football',  
'ballon_handball', 'ballon_rugby', 'ballon_volley', 'baseball', 'basketball']
```

Table 1: Parametres structurels fondamentaux du modele

Parametre	Symbole / Variable	Valeur
Dimension theorique de l’image d’entree	IMAGE_SIZE	64×64 pixels
Decoupage de la grille (cellules par dim)	CELL_PER_DIM	8×8 cellules
Nombre de boites englobantes par cellule	BOX_PER_CELL	1
Nombre de classes du probleme	NB_CLASSES	9
Resolution spatiale d’une cellule	PIX_PER_CELL	8 pixels

Sur le plan conceptuel, le probleme est formule selon une discretisation spatiale en grille bidimensionnelle de taille $S \times S$, ou $S = \text{CELL_PER_DIM} = 8$. Chaque image d’entree de taille 64×64 pixels est ainsi segmentee en 64 cellules elementaires de 8×8 pixels (PIX_PER_CELL).

Chaque cellule $i \in \{1, \dots, S^2\}$ a la responsabilite de predire au maximum $B = \text{BOX_PER_CELL} = 1$ boite englobante (bounding box). Le vecteur de prediction theorique associe a chaque cellule contient ainsi le score de confiance de presence d’un objet (C_i), les coordonnees relatives de la boite $\mathbf{b}_i = [x_i, y_i, w_i, h_i]^T$, et les probabilites conditionnelles de classes $P(\text{Classe}_k | \text{Objet})$.

Toutefois, pour surmonter les limites intrinseques d’un modele rudimentaire entraine *from scratch* sur des images de tres faible resolution (64×64), notre approche technique exploite la puissance du *Transfer Learning* via l’ecosysteme *Ultralytics*, tout en conservant la logique metier de classification a 9 classes.

2.2 Architecture du Reseau de Neurones : de la theorie a YOLO26n

Pour assurer une detection robuste et precise, nous utilisons l’architecture moderne **YOLO26n** (YOLO Nano, version 2026), un modele de detection d’objets *single-shot* optimise pour les performances en temps reel sur des architectures materielles contraintes.

L’architecture se decompose en trois blocs fonctionnels majeurs :

1. **Le Backbone (Extracteur de caracteristiques)** : Compose de convolutions de type DarkNet modifiees, integrant des blocs de goulot d’etranglement (Bottleneck CSP / C2f) et des connexions residuelles. Il reduit progressivement la resolution spatiale tout en augmentant la profondeur des canaux.
2. **Le Neck (Cou de liaison)** : Utilise une structure de type FPN (*Feature Pyramid Network*) combinee a un PANet (*Path Aggregation Network*). Ce bloc permet la fusion d’informations multi-echelles.

3. **La Head (Tete de detection)** : Il s’agit d’une tete *decoupled* (decoupee), separant explicitement les flux de regression des boites englobantes et les flux de classification. Ce modele est *anchor-free* (sans ancrs predefinies), predisant directement le centre de l’objet et les distances aux bords.

La couche de sortie du modele d’origine a ete modifiee pour s’adapter exactement a notre probleme a $N_c = 9$ classes au lieu des 80 classes du jeu de donnees COCO standard.

2.3 Pretraitement et Simplifications de la Base de Donnees

Bien que la structure logique du projet repose sur des images de 64×64 pixels, l’application directe de filtres de convolution profonds sur une telle resolution induirait un phenomene d’evanouissement de la resolution spatiale.

Pour y remedier, une **simplification et mise a l’echelle automatique** a ete integree :

- **Redimensionnement spatial (Upscaling)** : Les images sont reechantillonnees dynamiquement a une resolution de 640×640 pixels (`imgsz=640`) via une interpolation bilineaire lors du chargement. Cela permet de beneficier pleinement des poids pre-entraines.
- **Normalisation** : Les intensites de pixels $[0, 255]$ sont normalisees dans l’intervalle $[0.0, 1.0]$.
- **Fichier de configuration data.yaml** : Ce fichier centralise l’organisation du jeu de donnees.

2.4 Hyperparametres et Protocole d’Entrainement

L’entrainement a ete orchestre par le script Python fourni, dont l’appel a l’API `ultralalytics` configure explicitement l’environnement d’optimisation.

Table 2: Specifications exhaustives des hyperparametres d’entrainement

Hyperparametre	Valeur	Role et justification technique
<code>epochs</code>	100	Nombre maximal d’iterations completes sur le dataset.
<code>imgsz</code>	640	Resolution forcee des images en entree du reseau.
<code>device</code>	0	Execution sur le premier GPU (acceleration CUDA).
<code>workers</code>	2	Nombre de threads paralleles pour le chargement.
<code>cache</code>	True	Mise en cache des images en RAM pour acclereler les E/S.
<code>patience</code>	25	Seuil d’arret precoce si la perte de validation stagne.
<code>optimizer</code>	’auto’	Selection automatique (AdamW ou SGD) selon la topologie.
<code>batch</code>	-1	Auto-batching : calcul dynamique de la taille optimale.

2.5 Formulation Mathematique des Fonctions de Perte

La perte totale $\mathcal{L}_{\text{total}}$ est une combinaison lineaire de trois pertes distinctes :

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{dfl}} \quad (1)$$

Ou :

- **La perte de boite ($\mathcal{L}_{\text{box}}$)** : Repose sur la metrique *Complete IoU* (CIoU).

$$\mathcal{L}_{\text{box}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \quad (2)$$

- **La perte de classification ($\mathcal{L}_{\text{cls}}$)** : Utilise l'entropie croisee binaire (*BCE*) :

$$\mathcal{L}_{\text{cls}} = -\frac{1}{N_c} \sum_{k=1}^{N_c} [y_k \log(\hat{y}_k) + (1 - y_k) \log(1 - \hat{y}_k)] \quad (3)$$

- **La perte de focalisation de distribution ($\mathcal{L}_{\text{df}}$)** : Optimise la distribution des probabilites des coordonnees des boites.

2.6 Script d'Execution de l'Entrainement

Le code suivant decrit l'enchainement des instructions logicielles :

```

1 from ultralytics import YOLO
2
3 # 1. Initialisation du modele avec les poids pre-entraines
4 model = YOLO("yolo26n.pt")
5
6 # 2. Lancement du processus d'apprentissage
7 results = model.train(
8     data="./Projet-Apprentissage/data.yaml",
9     epochs=100,
10    device=0,
11    imgsz=640,
12    workers=2,
13    cache=True,
14    patience=25,
15    optimizer='auto',
16    batch=-1
17 )
18
19 # 3. Evaluation finale sur l'ensemble de validation/test
20 results = model.val()
```

Listing 1: Script de configuration et d'entrainement YOLO26n

Grace a ce formalisme d'entrainement, l'historique des pertes ainsi que l'evolution des metriques de precision et de rappel sont sauvegardes, fournissant la matiere premiere quantitative necessaire a l'analyse approfondie (Section 3).